Mermaid Flowchart Customization Guide

Deep dive into flowchart customization — shapes, colors, layout, styling, and advanced patterns.

Node Shapes

Every shape has semantic meaning. Choose based on the node’s purpose.

```
**Shape Syntax Quick Reference:**

| Shape | Syntax | Use Case |
|-------|--------|----------|
| Rectangle | `[text]` | Standard process |
| Rounded | `([text])` | Start/end points |
| Stadium | `([text])` | Same as rounded |
| Subroutine | `[[text]]` | Function call |
| Cylindrical | `[(text)]` | Database |
| Circle | `((text))` | Connection point |
| Double Circle | `(((text)))` | - |
| Asymmetric | `>text]` | Choice/flag |
| Rhombus | `{text}` | Decision |
| Hexagon | `{{text}}` | Preparation |
| Parallelogram (input) | `[/text/]` | Input data |
| Parallelogram (output) | `[\text\]` | Output data |
| Trapezoid | `[/text\]` | Manual operation |
| Inverted Trapezoid | `[\text/]` | - |

---

## Arrow Types and Line Styles

Arrows communicate relationships. Mix types to show different connection meanings.

![Mermaid Diagram 2](/tmp/tmp.MRDWJfYN0R/mermaid_2.png)

**Arrow Syntax:**

| Pattern | Code | Meaning |
|---------|------|---------|
| Solid line, arrow | `-->` | Standard flow |
| Dotted line, arrow | `-.->` | Optional/fallback |
| Thick line, arrow | `==>` | Primary/critical path |
| Solid line, no arrow | `---` | Association |
| Dotted line, no arrow | `-.-` | Weak association |
| Crossed | `--x` | Blocked |
| Circle ends | `o--o` | Bidirectional |
| Double arrow | `<-->` | Two-way communication |

**Adding Labels:**

![Mermaid Diagram 3](/tmp/tmp.MRDWJfYN0R/mermaid_3.png)

**Longer Arrows (add dashes):**

![Mermaid Diagram 4](/tmp/tmp.MRDWJfYN0R/mermaid_4.png)

---
```

```
## Color and Styling with classDef

`classDef` creates reusable style classes. Apply to multiple nodes for consistency.

### Basic classDef Pattern

![Mermaid Diagram 5](/tmp/tmp.MRDWJfYN0R/mermaid_5.png)

**classDef Syntax:**
```

classDef :,:,⋯ class

```

**Common Properties:**

| Property | Values | Example |
|----------|--------|---------|
| `fill` | Color hex/rgb | `fill:#ff6b6b` |
| `stroke` | Border color | `stroke:#c92a2a` |
| `stroke-width` | Border thickness | `stroke-width:3px` |
| `color` | Text color | `color:#fff` |
| `stroke-dasharray` | Dash pattern | `stroke-dasharray: 5 5` |
| `rx` | Corner radius | `rx:10px` |
| `ry` | Corner radius | `ry:10px` |

### Multiple Classes on One Node

![Mermaid Diagram 6](/tmp/tmp.MRDWJfYN0R/mermaid_6.png)

### Inline Styling (Quick One-offs)

![Mermaid Diagram 7](/tmp/tmp.MRDWJfYN0R/mermaid_7.png)

---

## Theme Customization

Apply global themes to entire diagrams.

### Pre-built Themes

![Mermaid Diagram 8](/tmp/tmp.MRDWJfYN0R/mermaid_8.png)

Available themes: `default`, `dark`, `forest`, `neutral`, `base`

### Custom Theme Variables

![Mermaid Diagram 9](/tmp/tmp.MRDWJfYN0R/mermaid_9.png)

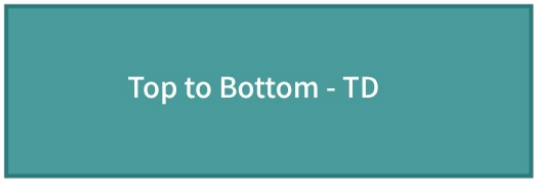
**Common Theme Variables:**

```javascript
{
  'primaryColor': '#color',        // Main nodes
  'primaryTextColor': '#color',    // Text on primary
  'primaryBorderColor': '#color',  // Border on primary
  'secondaryColor': '#color',      // Secondary nodes
  'tertiaryColor': '#color',       // Tertiary nodes
  'lineColor': '#color',           // Connector lines
  'fontSize': '16px',              // Base font size
  'fontFamily': 'sans-serif',      // Font family
  'background': '#color',          // Diagram background
```
```

```
  'mainBkg': '#color',              // Node background
  'textColor': '#color',            // Default text color
  'edgeLabelBackground': '#color'   // Label background
}
```

Layout and Direction

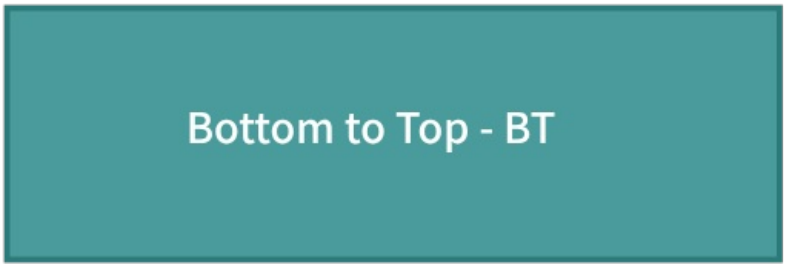
Control diagram orientation and flow.



Mermaid Diagram 10

Mermaid Diagram 11

Mermaid Diagram 12



Mermaid Diagram 13

Direction Codes:

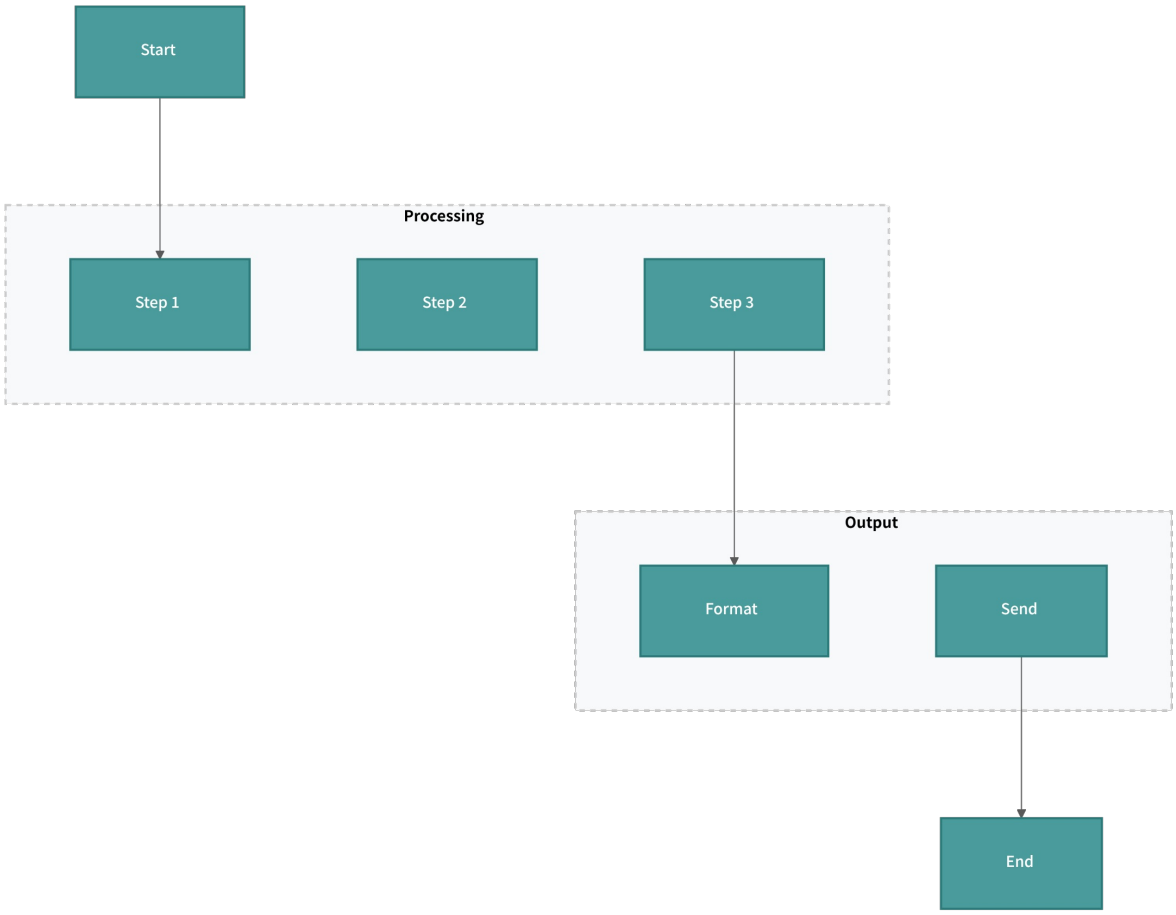
- `TD` or `TB` — Top Down / Top to Bottom (default)

- `LR` — Left to Right
- `RL` — Right to Left
- `BT` — Bottom to Top

Subgraphs for Grouping

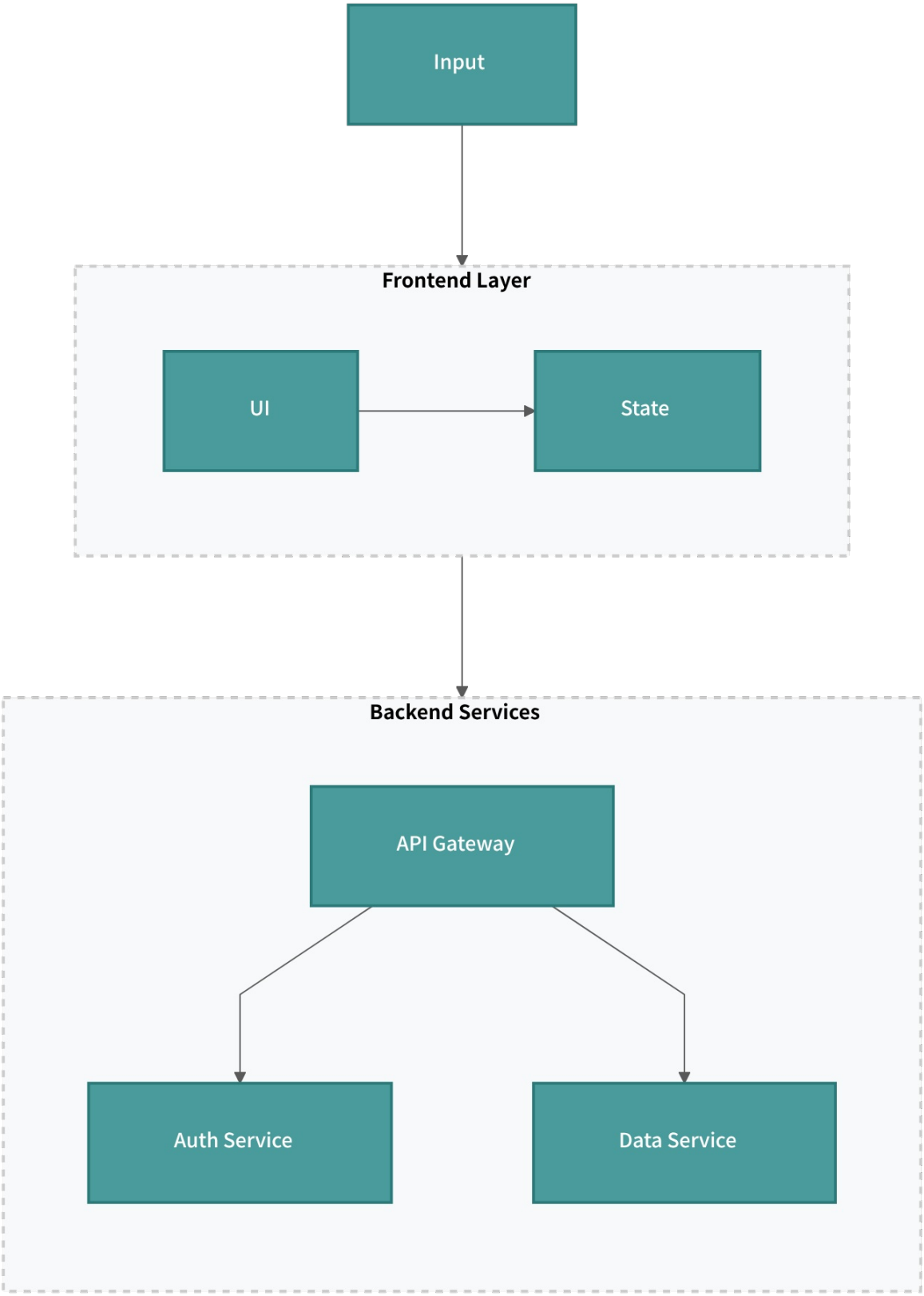
Subgraphs visually group related nodes and can have their own direction.

Basic Subgraph



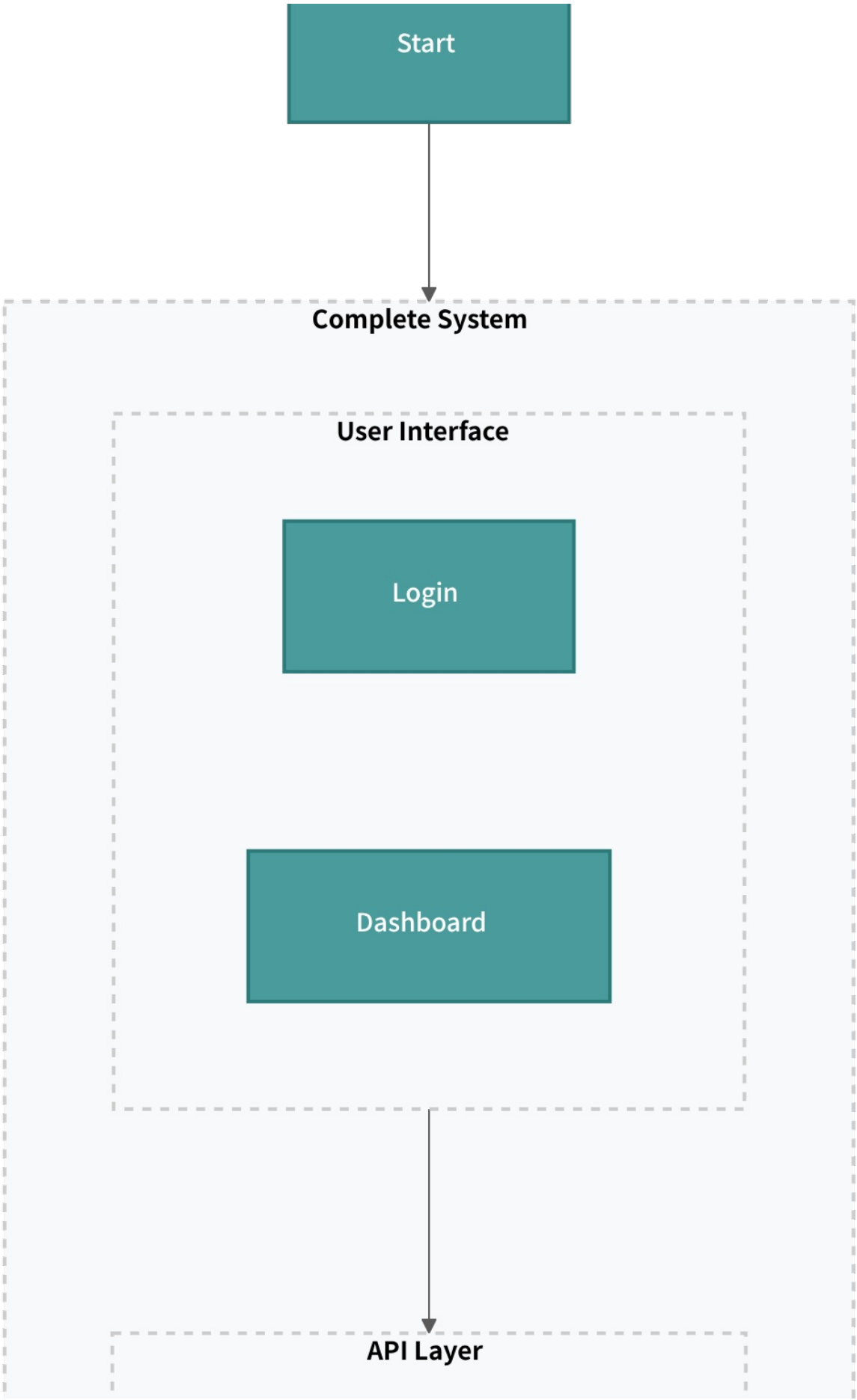
Mermaid Diagram 14

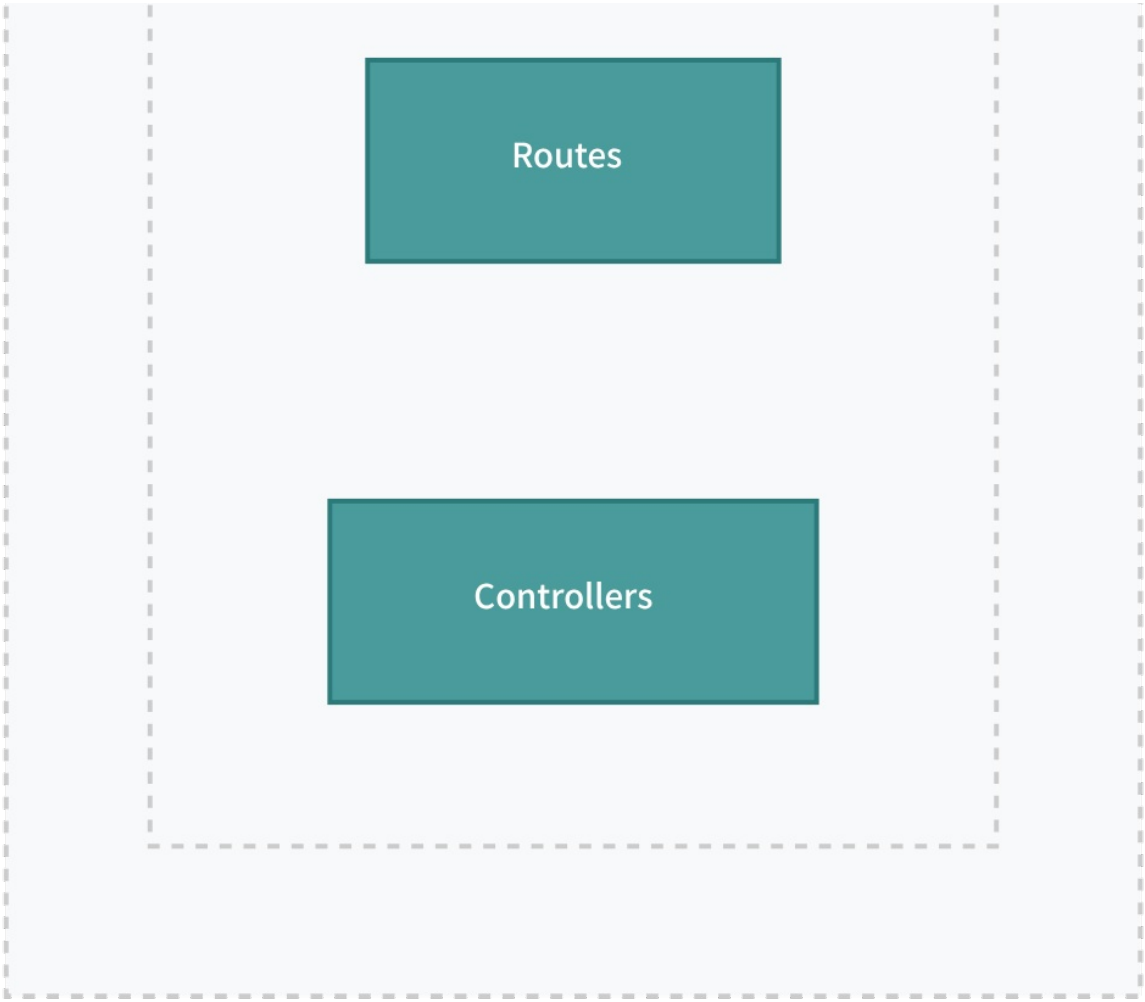
Named Subgraphs with Custom Direction



Mermaid Diagram 15

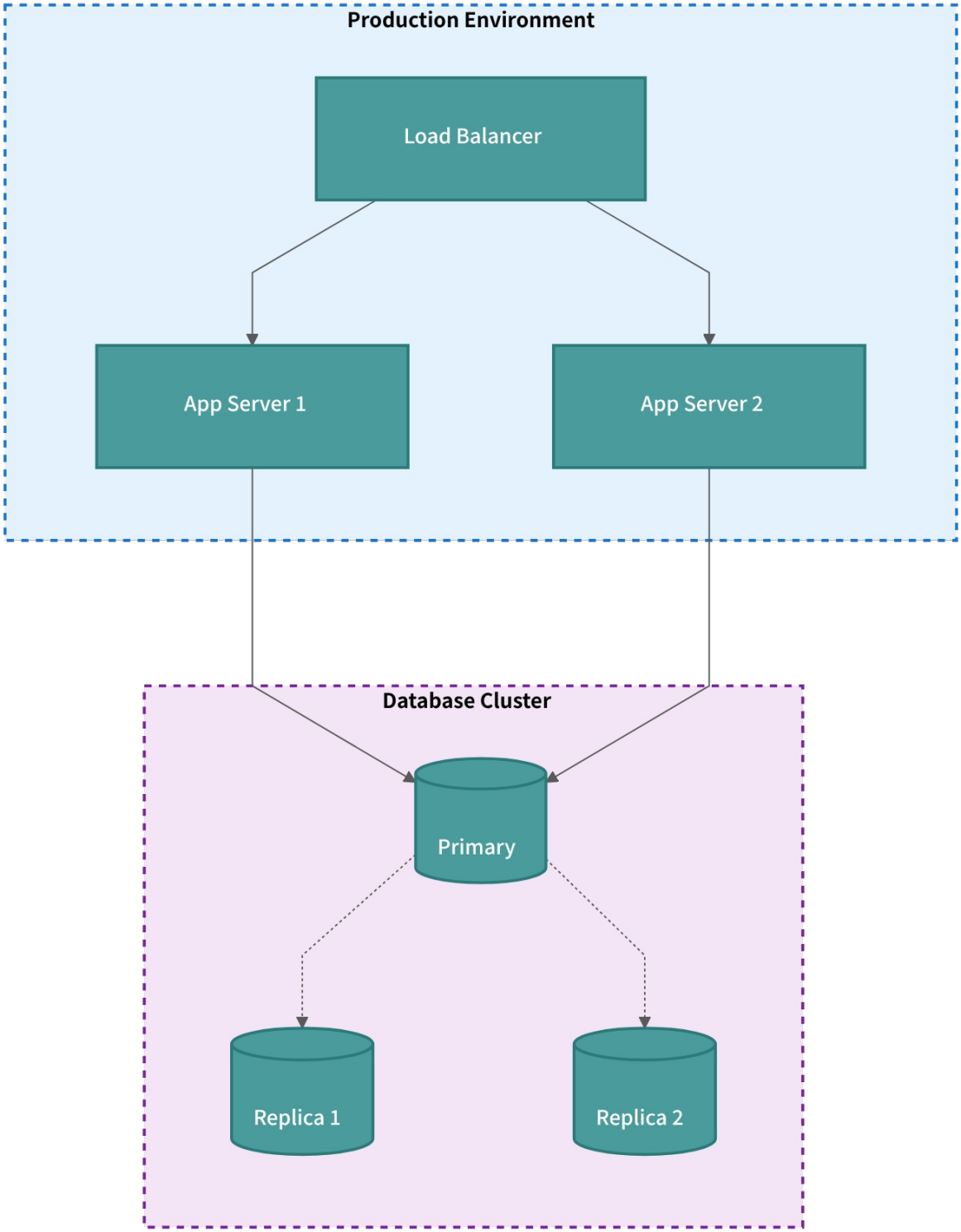
Nested Subgraphs


Start
Complete System
User Interface
Login
Dashboard
API Layer



Mermaid Diagram 16

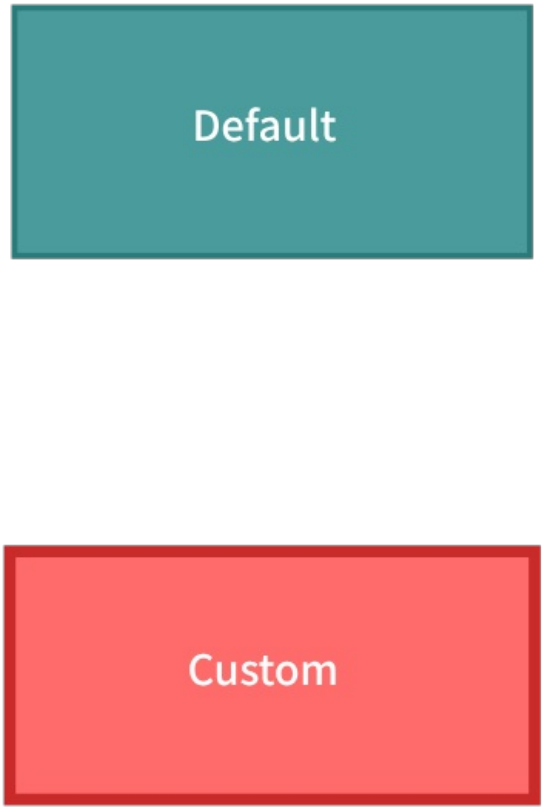
Styled Subgraphs



Mermaid Diagram 17

Advanced Styling Patterns

Individual Node Styling



Mermaid Diagram 18

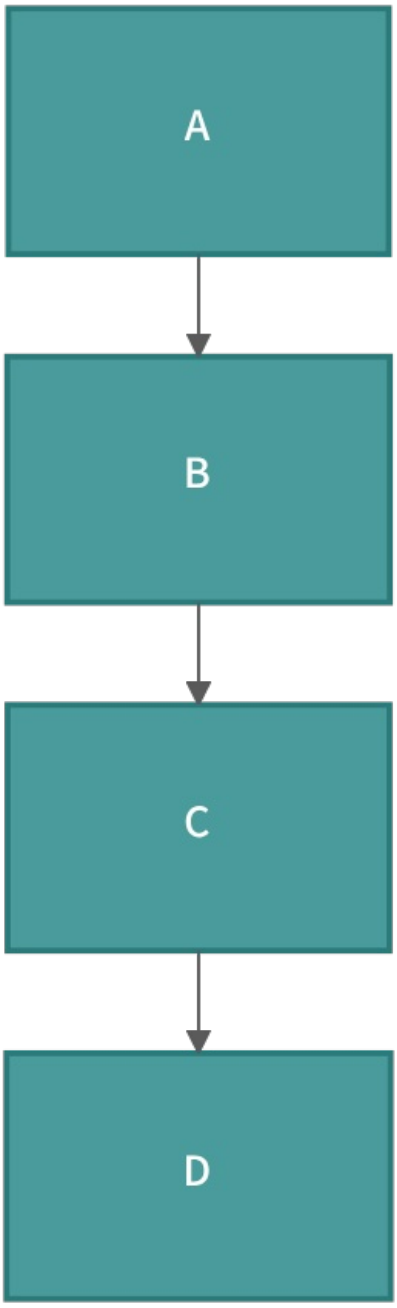
Link Styling by Index

Mermaid Diagram 19

Link Styling Properties:

```
linkStyle <index> stroke:<color>,stroke-width:<width>,stroke-dasharray:<pattern>
```

Default Styles for All Nodes/Links



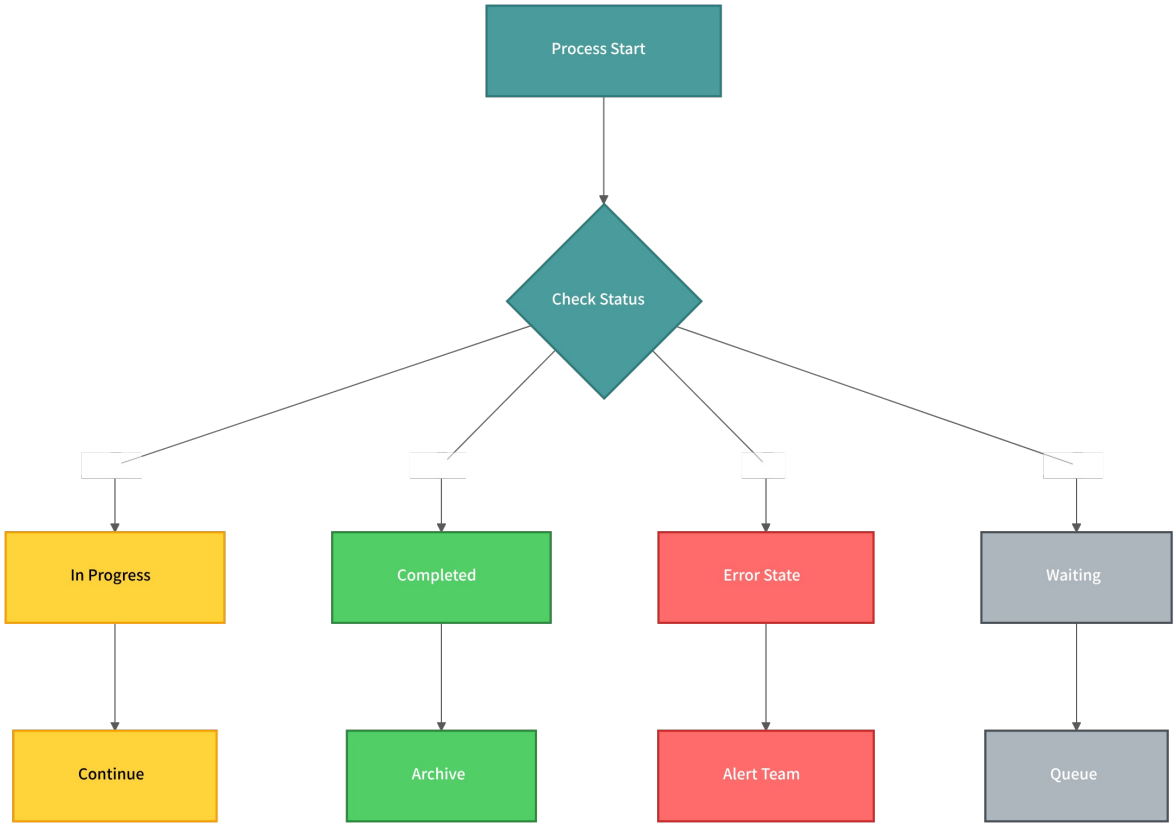
Mermaid Diagram 20

Curve Options:

- `basis` — Smooth curves
- `linear` — Straight lines
- `step` — Step-like connections
- `stepBefore` — Step before
- `stepAfter` — Step after

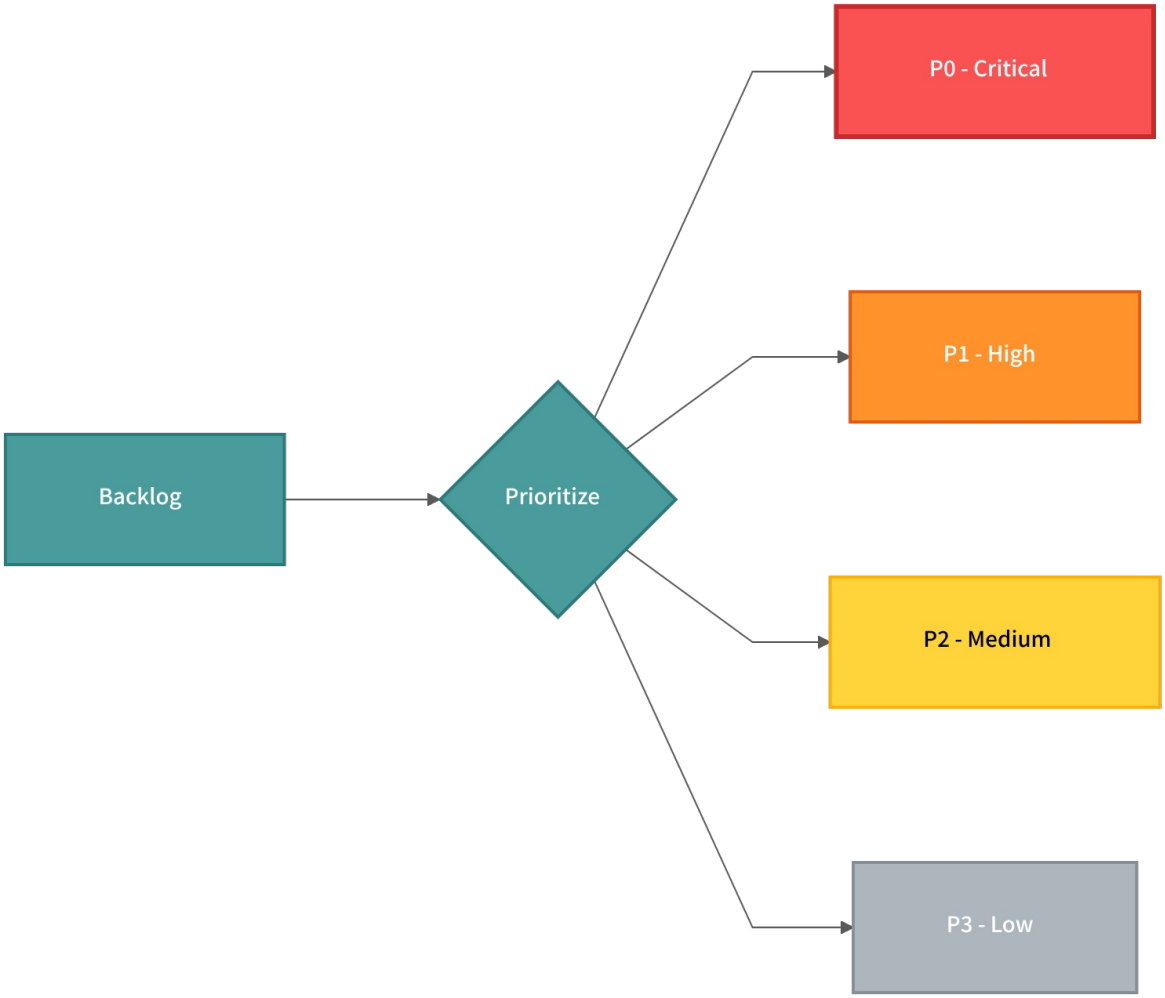
Practical Styling Patterns

Status Indicators



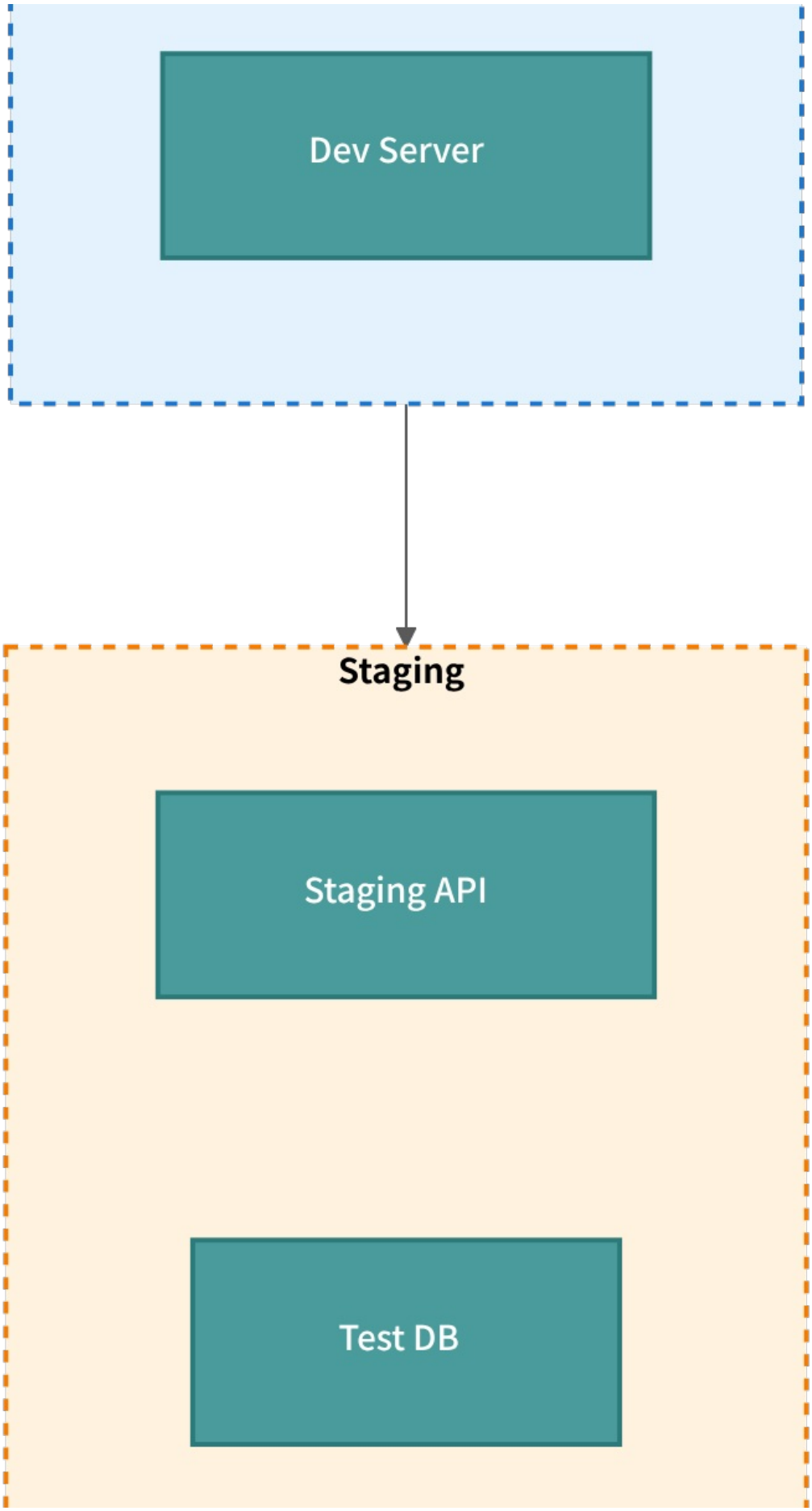
Mermaid Diagram 21

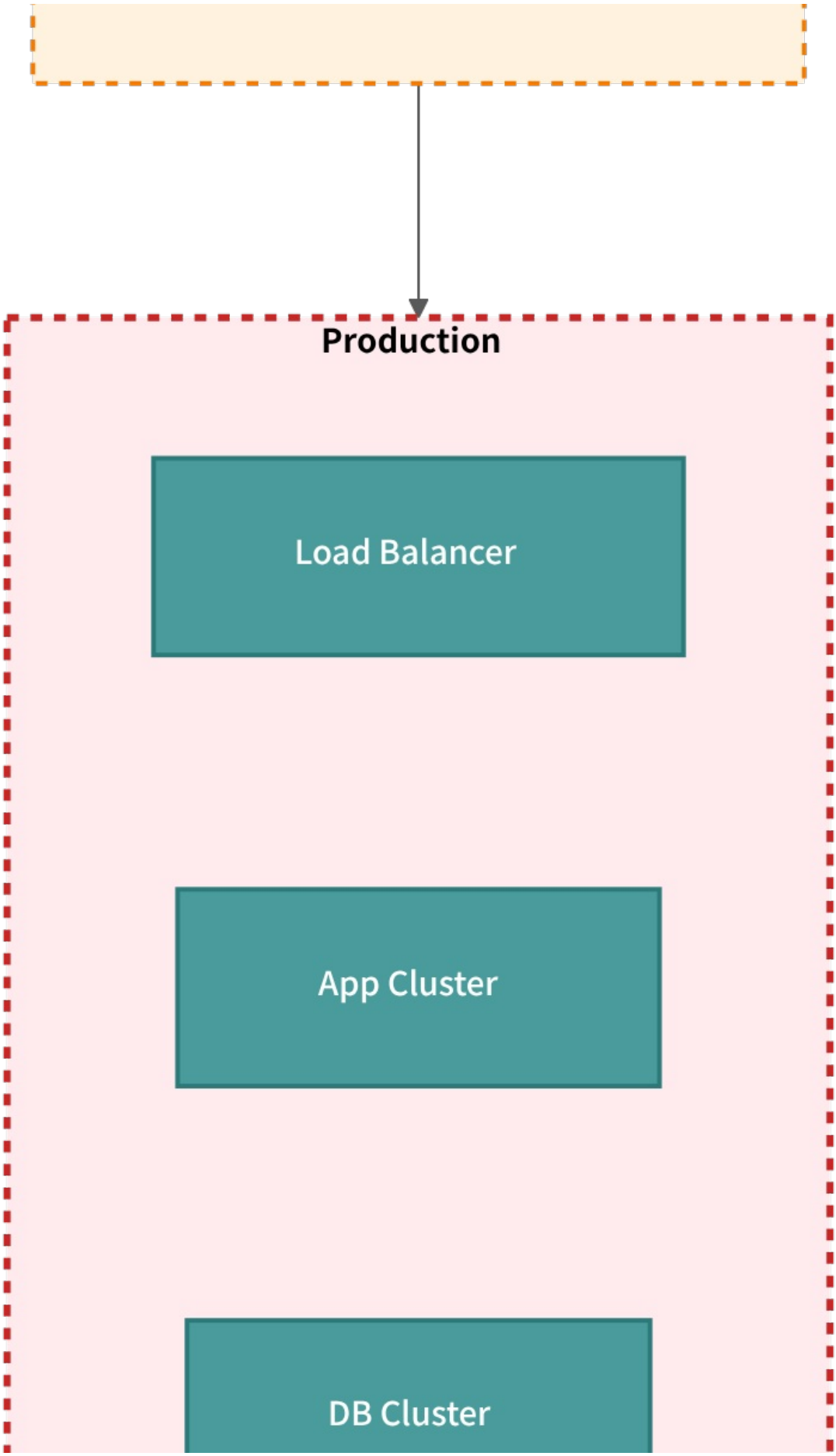
Priority Levels

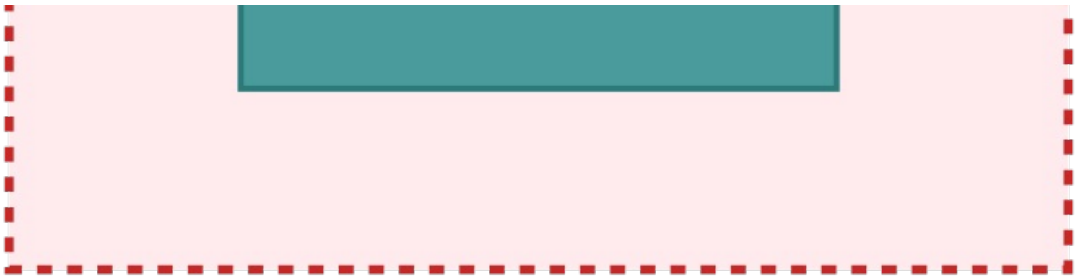


Mermaid Diagram 22

Environment Layers

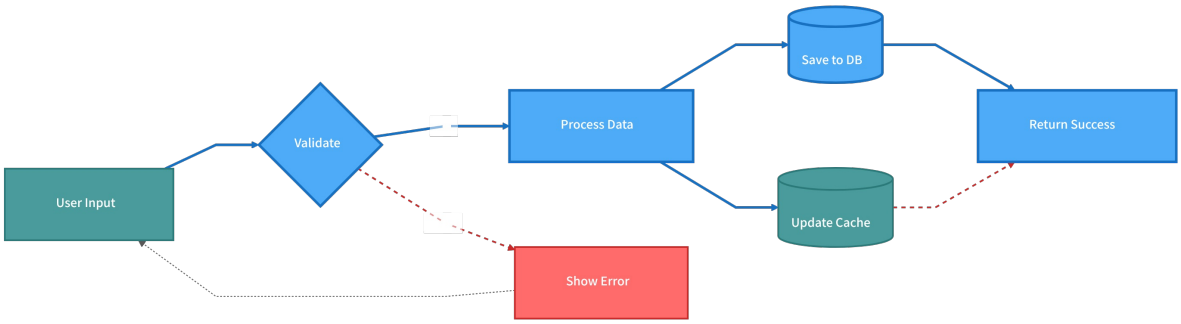

Dev Server
Staging
Staging API
Test DB


Production
Load Balancer
App Cluster
DB Cluster



Mermaid Diagram 23

Data Flow with Emphasis



Mermaid Diagram 24

Interactive Features

Clickable Nodes

```
**Click Syntax:**
```

click “” “” [_blank/_self/_parent/_top] click href “” “” click call () “”

```
### Tooltips (via title attribute)

```mermaid

---

## Layout Optimization

### Managing Complex Layouts

**Problem: Messy crossing lines**

![Mermaid Diagram 27](/tmp/tmp.MRDWJfYN0R/mermaid_27.png)

**Solution: Use invisible nodes for routing**

![Mermaid Diagram 28](/tmp/tmp.MRDWJfYN0R/mermaid_28.png)
```
```

Rank Separation Control

Force nodes onto the same level:

![Mermaid Diagram 29](/tmp/tmp.MRDWJfYN0R/mermaid_29.png)

Real-World Examples

CI/CD Pipeline

![Mermaid Diagram 30](/tmp/tmp.MRDWJfYN0R/mermaid_30.png)

Authentication Flow with Error Handling

![Mermaid Diagram 31](/tmp/tmp.MRDWJfYN0R/mermaid_31.png)

Debugging Tips

1. **Test incrementally** — Add nodes/styling one at a time
2. **Use mermaid.live** for real-time preview
3. **Comment out sections** to isolate issues:

![Mermaid Diagram 32](/tmp/tmp.MRDWJfYN0R/mermaid_32.png)

4. **Validate class names** — must be alphanumeric
5. **Check link indices** — `linkStyle 0` is the first arrow, counting from top

---

Cheat Sheet: Property Values

Colors:
- Hex: `#ff6b6b`
- RGB: `rgb(255, 107, 107)`
- RGBA: `rgba(255, 107, 107, 0.5)`
- Named: `red`, `blue`, `green`

Sizes:
- Pixels: `2px`, `10px`
- Percent: `50%`
- Em: `1.5em`

Stroke Patterns:
- Solid: `stroke-width:2px`
- Dashed: `stroke-dasharray: 5 5`
- Dotted: `stroke-dasharray: 1 3`
- Long dash: `stroke-dasharray: 10 5`

Font Families:
- `monospace`
- `sans-serif`
- `serif`
- `'Inter', system-ui, sans-serif`

Common Color Palettes

Status Colors (Friendly)

Success: fill:#51cf66,stroke:#2b8a3e,color:#fff Warning: fill:#ffd43b,stroke:#f59f00,color:#000 Error: fill:#ff6b6b,stroke:#c92a2a,color:#fff Info: fill:#4dabf7,stroke:#1971c2,color:#fff Neutral: fill:#adb5bd,stroke:#495057,color:#fff

```
### Priority Levels
```

P0 Critical: fill:#fa5252,stroke:#c92a2a,color:#fff P1 High: fill:#ff922b,stroke:#e8590c,color:#fff P2 Medium: fill:#ffd43b,stroke:#fab005,color:#000 P3 Low: fill:#94d82d,stroke:#5c940d,color:#000

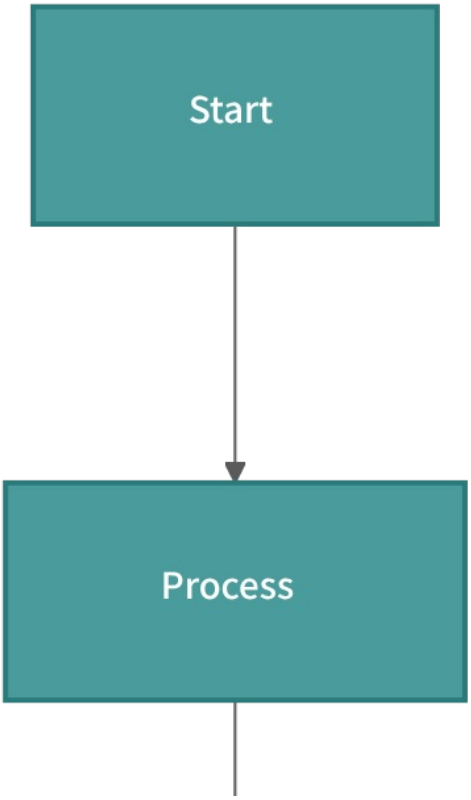
```
### Environment Tiers
```

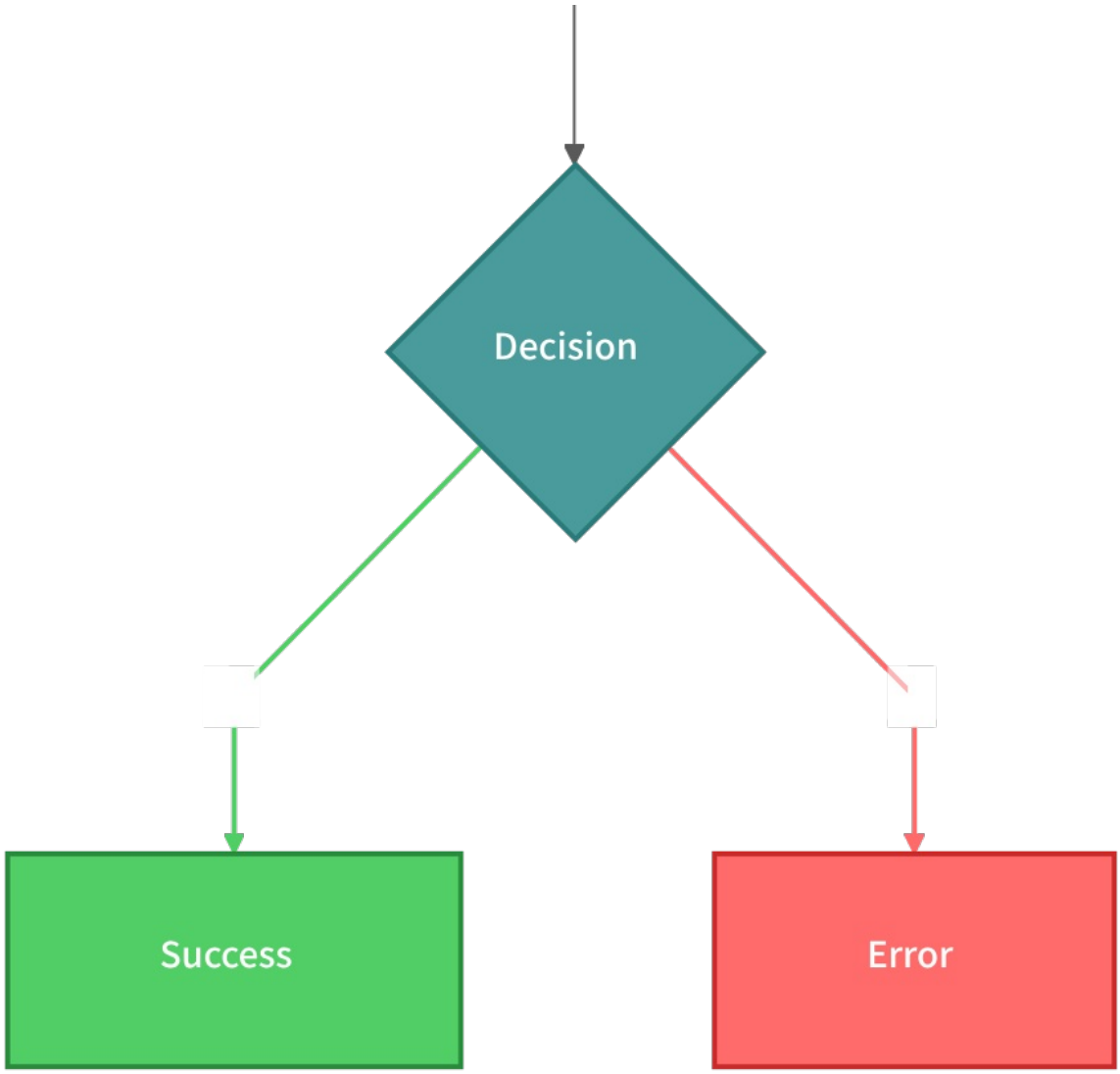
Dev: fill:#e3f2fd,stroke:#1976d2,color:#000 Staging: fill:#fff3e0,stroke:#f57c00,color:#000 Production: fill:#ffebee,stroke:#c62828,color:#000 ```

Quick Customization Workflow

1. **Build structure** (nodes + arrows)
2. **Define classDef** for common styles
3. **Apply classes** to nodes
4. **Style links** if needed
5. **Add subgraphs** for grouping
6. **Apply theme** if needed
7. **Test and refine**

Template:





Mermaid Diagram 33